

# SIMATS Engineering

## SIMATS Online Programming Lab — Python Programming

|                 |                                                                    |
|-----------------|--------------------------------------------------------------------|
| Course Name     | Python Programming                                                 |
| Course Code     | CSA0804                                                            |
| Name            | Monish.L                                                           |
| Register Number | 1972260035                                                         |
| Topic           | CO2-AT2-SCENARIO BASED ASSESSMENT                                  |
| Question        | Q1 — Scenario 1: Intelligent Customer Feedback Analysis (25 Marks) |

---

### Scenario 1: Intelligent Customer Feedback Analysis [25 Marks]

An e-commerce platform collects large volumes of customer feedback in unstructured textual form. The feedback often contains inconsistent capitalization, punctuation, and unnecessary spacing. The system must preprocess the text by normalizing case and removing unwanted characters. It should then identify specific keywords that indicate positive or negative sentiment. The program must count the occurrences of such keywords efficiently. Based on the counts, the system should classify the feedback into categories. The classification logic must be implemented using appropriate control flow constructs. The solution should be modularized using functions for preprocessing and analysis. It should also ensure scalability for handling multiple feedback entries.

#### Task

Design a Python program to implement this intelligent feedback analysis system, demonstrating the following key skills:

- String normalization: `lower()`, `strip()`, `replace()`
- Keyword counting using loops and conditionals
- Sentiment classification using `if-elif-else`
- Modular design with functions
- Handling multiple feedback entries using loops

## Approach / Design

- **Preprocessing (preprocess\_text):** converts text to lowercase, strips leading/trailing whitespace, removes punctuation, and collapses extra internal spaces so keyword matching is consistent.
- **Keyword lists:** two lists — POSITIVE\_KEYWORDS and NEGATIVE\_KEYWORDS — hold the sentiment-indicating words.
- **Keyword counting (count\_keywords):** loops through the cleaned words and uses a conditional check to count matches against a keyword list.
- **Sentiment classification (classify\_sentiment):** uses if-elif-else on the positive/negative counts to decide Positive, Negative, or Neutral.
- **Category classification (classify\_category):** checks for domain-specific keywords (delivery, quality, service) to tag the feedback topic.
- **Batch handling (analyze\_all\_feedback):** loops over any number of feedback strings, so the design scales to large volumes of feedback.
- **Reporting (print\_report):** prints a per-entry breakdown plus overall sentiment statistics.

## Python Program

```
"""
C02-AT2 - Scenario Based Assessment
Scenario 1: Intelligent Customer Feedback Analysis
-----
An e-commerce platform collects unstructured customer feedback.
This program:
    1. Normalizes text (case, punctuation, spacing)
    2. Counts occurrences of positive/negative keywords
    3. Classifies each feedback as Positive / Negative / Neutral
    4. Handles multiple feedback entries using loops
    5. Is fully modularized using functions
"""

import string

# -----
# 1. Keyword lists used for sentiment detection
# -----
POSITIVE_KEYWORDS = [
    "good", "great", "excellent", "amazing", "love",
    "happy", "satisfied", "fantastic", "awesome", "best"
]

NEGATIVE_KEYWORDS = [
    "bad", "poor", "terrible", "worst", "hate",
    "disappointed", "broken", "late", "awful", "damaged"
]

# -----
# 2. Preprocessing function
# -----
def preprocess_text(text):
    """
    Normalizes raw feedback text:
    - Converts to lowercase
    - Strips leading/trailing whitespace
    - Removes punctuation
    - Collapses multiple spaces into a single space
    """
```

```

    text = text.lower().strip()
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = " ".join(text.split()) # removes extra internal spaces
    return text

# -----
# 3. Keyword counting function
# -----
def count_keywords(words, keyword_list):
    """
    Counts how many times keywords from keyword_list appear
    in the given list of words using a loop and conditional check.
    """
    count = 0
    for word in words:
        if word in keyword_list:
            count += 1
    return count

# -----
# 4. Sentiment classification function
# -----
def classify_sentiment(pos_count, neg_count):
    """
    Classifies sentiment using if-elif-else control flow
    based on positive and negative keyword counts.
    """
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

# -----
# 5. Category classification function
# -----
def classify_category(cleaned_text):
    """
    Classifies feedback into a broad category based on
    keyword presence (Delivery, Product Quality, Service, General).
    """
    if any(word in cleaned_text for word in ["delivery", "shipping", "late", "delayed"]):
        return "Delivery"
    elif any(word in cleaned_text for word in ["quality", "broken", "damaged", "defective"]):
        return "Product Quality"
    elif any(word in cleaned_text for word in ["staff", "service", "support", "response"]):
        return "Customer Service"
    else:
        return "General"

# -----
# 6. Single feedback analysis function
# -----
def analyze_feedback(raw_text):
    """
    Runs the full pipeline on one feedback entry and
    returns a dictionary summarizing the analysis.
    """
    cleaned = preprocess_text(raw_text)
    words = cleaned.split()

    pos_count = count_keywords(words, POSITIVE_KEYWORDS)
    neg_count = count_keywords(words, NEGATIVE_KEYWORDS)
    sentiment = classify_sentiment(pos_count, neg_count)

```

```

category = classify_category(cleaned)

return {
    "original": raw_text,
    "cleaned": cleaned,
    "positive_count": pos_count,
    "negative_count": neg_count,
    "sentiment": sentiment,
    "category": category
}

# -----
# 7. Batch processing function (handles multiple entries)
# -----
def analyze_all_feedback(feedback_list):
    """
    Iterates through a list of feedback entries, analyzes each
    one, and returns a list of result dictionaries. Designed to
    scale to any number of feedback entries.
    """
    results = []
    for entry in feedback_list:
        result = analyze_feedback(entry)
        results.append(result)
    return results

# -----
# 8. Report/display function
# -----
def print_report(results):
    """
    Prints a formatted summary report for all analyzed feedback,
    plus overall statistics.
    """
    pos_total = neg_total = neu_total = 0

    print("=" * 70)
    print("CUSTOMER FEEDBACK ANALYSIS REPORT")
    print("=" * 70)

    for idx, res in enumerate(results, start=1):
        print(f"\nFeedback #{idx}")
        print(f"  Original : {res['original']}")
        print(f"  Cleaned  : {res['cleaned']}")
        print(f"  Positive keyword count : {res['positive_count']}")
        print(f"  Negative keyword count : {res['negative_count']}")
        print(f"  Sentiment : {res['sentiment']}")
        print(f"  Category  : {res['category']}")

        if res["sentiment"] == "Positive":
            pos_total += 1
        elif res["sentiment"] == "Negative":
            neg_total += 1
        else:
            neu_total += 1

    total = len(results)
    print("\n" + "-" * 70)
    print("SUMMARY")
    print("-" * 70)
    print(f"Total feedback entries : {total}")
    print(f"Positive : {pos_total}   ({(pos_total/total)*100:.1f}%)")
    print(f"Negative : {neg_total}   ({(neg_total/total)*100:.1f}%)")
    print(f"Neutral  : {neu_total}   ({(neu_total/total)*100:.1f}%)")

# -----

```

```
# 9. Main driver
# -----
def main():
    # Sample unstructured customer feedback (multiple entries)
    feedback_list = [
        " The PRODUCT quality is AMAZING!! Loved it a lot... ",
        "Delivery was very LATE, and the package was Damaged.",
        " average experience, nothing special. ",
        "Excellent customer SERVICE!! The staff was so helpful & Great!!",
        "Worst purchase ever... product broken on arrival, terrible!!",
    ]

    results = analyze_all_feedback(feedback_list)
    print_report(results)

if __name__ == "__main__":
    main()
```

## Sample Output

```
=====
CUSTOMER FEEDBACK ANALYSIS REPORT
=====

Feedback #1
  Original : The PRODUCT quality is AMAZING!! Loved it a lot...
  Cleaned  : the product quality is amazing loved it a lot
  Positive keyword count : 1
  Negative keyword count : 0
  Sentiment : Positive
  Category  : Product Quality

Feedback #2
  Original : Delivery was very LATE, and the package was Damaged.
  Cleaned  : delivery was very late and the package was damaged
  Positive keyword count : 0
  Negative keyword count : 2
  Sentiment : Negative
  Category  : Delivery

Feedback #3
  Original : average experience, nothing special.
  Cleaned  : average experience nothing special
  Positive keyword count : 0
  Negative keyword count : 0
  Sentiment : Neutral
  Category  : General

Feedback #4
  Original : Excellent customer SERVICE!! The staff was so helpful & Great!!
  Cleaned  : excellent customer service the staff was so helpful great
  Positive keyword count : 2
  Negative keyword count : 0
  Sentiment : Positive
  Category  : Customer Service

Feedback #5
  Original : Worst purchase ever... product broken on arrival, terrible!!
  Cleaned  : worst purchase ever product broken on arrival terrible
  Positive keyword count : 0
  Negative keyword count : 3
  Sentiment : Negative
  Category  : Product Quality

-----
SUMMARY
-----
Total feedback entries : 5
Positive : 2 (40.0%)
Negative : 2 (40.0%)
Neutral  : 1 (20.0%)
```

## Conclusion

The program successfully preprocesses raw, inconsistent feedback text, counts positive and negative keyword occurrences using loops and conditionals, classifies each entry's sentiment using if-elif-else logic, and produces a summary report. Because the analysis logic is wrapped in reusable functions and driven by a loop over a list, the same code scales to any number of feedback entries without modification.